Twice As Nice?

Nimish Chandiramani

If you were to read the lay (and, unfortunately, very popular) definition of 64-bit computing—"A 64-bit computer can process twice as much information as a 32-bit computer"—you'd be expecting programs to run twice as fast and games to look twice as good. And yet, not much has changed in the past few years—well, aside from all the hype dying down a bit.

So what do we expect from 64-bit computing when it does "arrive"?

The Bits That Matter

why do we speak of computers (processors, to be precise), in terms of bits? What do the 64 bits in a 64-bit processor actually *do*? On the processor, this means that the *registers* on the processor—which act as temporary storage areas—can store 64 bits of data at a time. It also means that the Arithmetic and Logic Unit (ALU) of the processor can work with 64-bit numbers—which is how the generalised definition comes about. The truth, however, is not as glamourous or exhilarating.

To understand, let's consider a system we understand better—decimal numbers. If the Powers That Be told you that you're allowed only to use one digit, you'd only be able to represent the first ten integers—0 to 9. Now, if you were given another digit, think of the possibilities—you can represent a hundred numbers now, all the way from 0 to 99. Let's take another look at the definition now—*twice* as much information.

A random number like 43 isn't twice as much information as, say, the number 6, is it? It's just different information—the extra digit has only multiplied the number of possibilities: one digit gave you ten (10^1) numbers, two give you a hundred (10^2), three will give you a thousand (10^3) and so on.

It's the same with binary digits, only it's a factor of two we're talking about—one bit gives you two numbers (2^1), two gives you four (2^2), three, eight (2^3), and onwards. With 32 bits, you can represent around 4.3 billion (2^{32}) numbers, and with 64 bits, you multiply that by another 4.3 billion. So, you see, 64-bit computers don't process twice as much information, but they can represent 4.3 billion times as many numbers as 32-bit computers.

But what does it do with all those numbers?

Remember This?

Since you've likely been subjected to a lot of pro-64 propaganda, you'll no longer recall the most popular problem with 32-bit computers—they can only use 4 GB of memory. With each memory location a byte long, that gives us 4 GB of memory. With 64-bit memory addresses, that becomes a theoretical 18 million terabytes—16 exabytes—of addressable memory!

Even six months ago, it seemed unlikely that you'd invest in 4 GB of RAM—even if you could afford more—but that's changed now. Try to run the 32-bit version of Windows XP (which is what you'll most likely have anyway) with that, and you'll see that it only shows you 3.5-odd GB available. Moreover, no application will ever be allocated more than 2 GB of RAM—it's just the way Windows is built.

To be able to use more than 4 GB of RAM—and that could be sooner than you think—you obviously need a 64-bit computer, and an operating system that supports 64-bit computing.

And what do you do with all that memory? Well, if you're a regular PC user—using the Internet, an office suite, IM and a bit of casual gaming—nothing. You'll have bragging rights, but anything more than 2 GB (even if you're running Vista) is wasted on you. If, however, you use 3D design and / or audio editing applications, raising your ceiling above 4 GB lets you load larger textures and more sound samples into the system memory, making your programs rely less on virtual memory, and thus much more responsive. Ditto if you're running a database or Web server.

But more memory isn't the only good thing about 64-bit computing.

Doing The Math

A 32-bit computer is pretty much similar to a 10-digit calculator. When you exceed the maximum number that can be represented with these 32 bits, the processor gives you an inaccurate result and tells you that there's been an *overflow*. So have we been restricting ourselves to 32-bit numbers all this time?

Unfortunately, to develop applications that need to deal with large numbers, programmers couldn't afford to wait for more range, so they devised a way that'll let processors deal with 64-bit numbers, 32 bits at a time. To store a 64-bit number, a 32-bit processor would take two clock cycles—64-bit processors do it in one. It's the same with any operation—load, add, subtract, multiply—and when you must deal with many 64-bit numbers, the disadvantages of 32-bit computing become more obvious.